

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Berbasis Framework

Pertemuan: Client Side Rendering & Data Fetching

Framework: Next.js

Library Tambahan: SWR

A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menjelaskan konsep Client Side Rendering (CSR).
 2. Mengimplementasikan data fetching menggunakan useEffect.
 3. Membuat Skeleton Loading menggunakan conditional rendering.
 4. Menggunakan library SWR untuk optimasi data fetching.
 5. Mengimplementasikan caching pada aplikasi berbasis framework.
-

B. Dasar Teori

Client Side Rendering (CSR)

Client Side Rendering adalah proses rendering UI yang dilakukan di sisi browser (client).

Ciri-ciri:

- HTML awal kosong.
- Data diambil setelah halaman dimuat.
- Ada delay sebelum data tampil.
- Cocok untuk aplikasi interaktif.

Alur CSR:

Server kirim HTML kosong + JS

↓

Browser menjalankan JS

↓

Fetch data dari API

↓

Render UI di client

2 Data Fetching dengan useEffect (Contoh Penggunaan CSR)

Contoh pola CSR dasar:

```
useEffect(() => {  
  fetch('/api/products')  
    .then(res => res.json())  
    .then(data => setProducts(data))  
}, [])
```

Karakteristik:

- Data diambil di sisi client
 - Akan ada loading state
 - Perlu conditional rendering
-

Skeleton Loading

Skeleton digunakan untuk:

- Menghindari tampilan kosong
- Memberikan feedback visual
- Meningkatkan UX

Contoh konsep:

```
{products.length > 0 ? (  
  products.map(...)  
) : (  
  <Skeleton />  
)}
```

Data Fetching dengan SWR

SWR adalah React Hooks untuk data fetching dengan caching otomatis.

Instalasi:

npm install swr

Contoh penggunaan:

```
import useSWR from 'swr'
```

```
const fetcher = (url: string) => fetch(url).then(res => res.json())
```

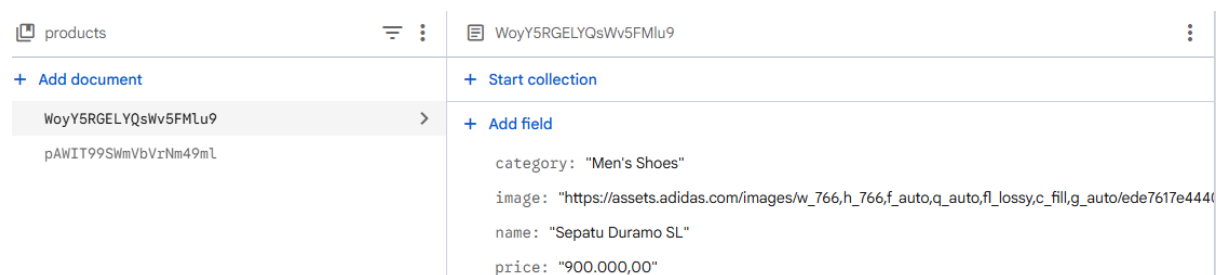
```
const { data, error, isLoading } = useSWR('/api/products', fetcher)
```

Keunggulan:

- Caching otomatis
- Revalidation
- Handling loading & error lebih sederhana
- Lebih clean dibanding useEffect manual

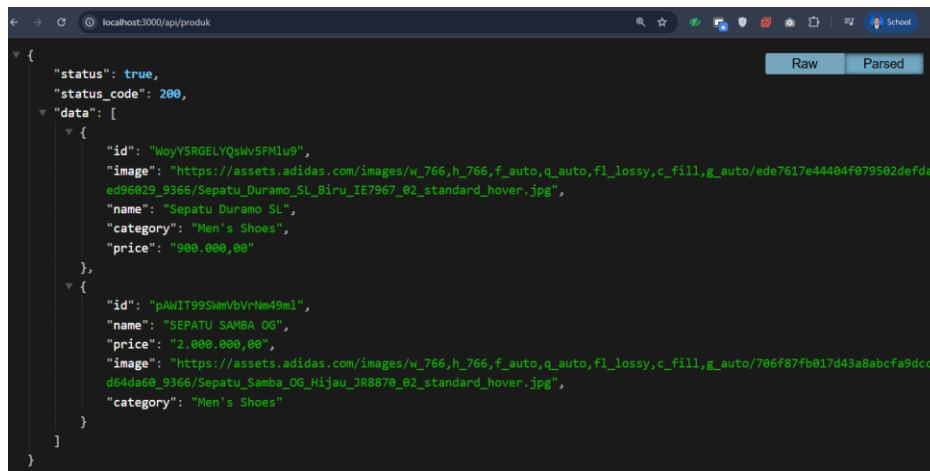
✂ C. Langkah Praktikum

Buka firebase dan rubah fieldnya serta tambahkan beberapa field seperti pada gambar dan untuk image bisa gunakan url pada image toko-toko sepatu (copy image address). Selain itu tambah juga documentnya sehingga ada 2 document



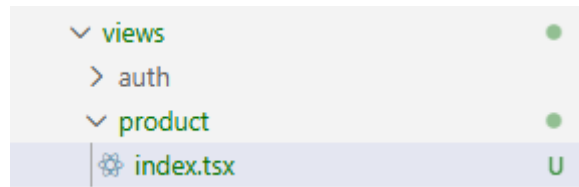
◆ Bagian 1 – Setup Data Produk

1. Siapkan project Next.js.
2. Buat endpoint API /api/products.
3. Pastikan data memiliki:
 - id
 - name
 - category
 - price
 - image
4. jalankan browser <http://localhost:3000/api/produk>



◆ Bagian 2 – Implementasi CSR dengan useEffect

1. Membuat file index.tsx pada folder views/products



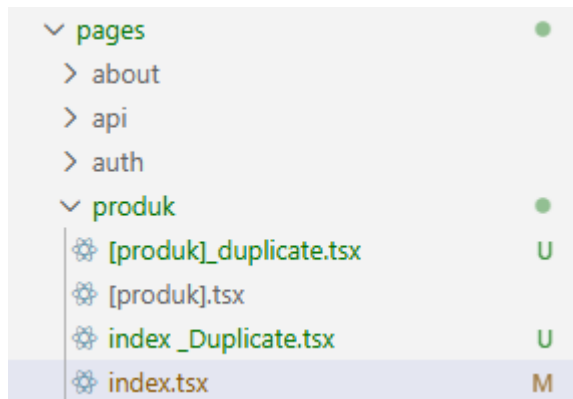
2. Modifikasi index.tsx

```

7.praktikum-Client_side rendering > my-app > src > pages > views > product > index.tsx > ...
1
  Windsurf: Refactor | Explain
2 type ProductType = {
3   id: string;
4   name: string;
5   price: number;
6   image: string;
7   category: string;
8 };
9
10 Windsurf: Refactor | Explain | Generate JSDoc | X
11 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
12   return (
13     <div>
14       <h1>Daftar Produk</h1>
15       {products.map((products: ProductType) => (
16         <div key={products.id}>
17           <h2>nama :{products.name}</h2>
18           <p>Harga: {products.price}</p>
19           <img src={products.image} alt={products.name} width={200} />
20           <p>kategori: {products.category}</p>
21         </div>
22       ))}
23     </div>
24   );
25 }
26 export default TampilanProduk;
27

```

3. Buka file index.tsx pada pages/produk/



4. Modifikasi index.tsx pada pages/produk/

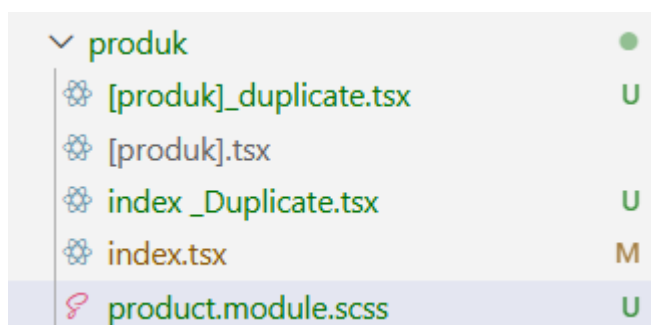
7.praktikum-Client_side rendering > my-app > src > pages > produk > index.tsx > kategori > useEffect() callback > then() callback

```
1 import { useRouter } from "next/router";
2 import { useEffect, useState } from "react";
3 import TampilanProduk from "../views/product";
4
5 const kategori = () => {
6   // const [isLogin, setIsLogin] = useState(false);
7   // const { push } = useRouter();
8   const [products, setProducts] = useState([]);
9   // console.log("products:", products);
10  // useEffect(() => {
11  //   if (!isLogin) {
12  //     push("/auth/login");
13  //   }
14  // }, []);
15
16  useEffect(() => {
17    fetch("/api/produk")
18      .then((response) => response.json())
19      .then((responseData) => {
20        setProducts(responseData.data);
21        // console.log("Data produk:", responseData.data);
22      })
23      .catch((error) => {
24        console.error("Error fetching produk:", error);
25      });
26  }, []);
27
28  return (
29    <div>
30      <TampilanProduk products={products} />
31    </div>
32  );
33 };
34
35 export default kategori;
```

5. Jalankan browser <http://localhost:3000/produk>



6. Pada folder produk buat file produk.modules.scss



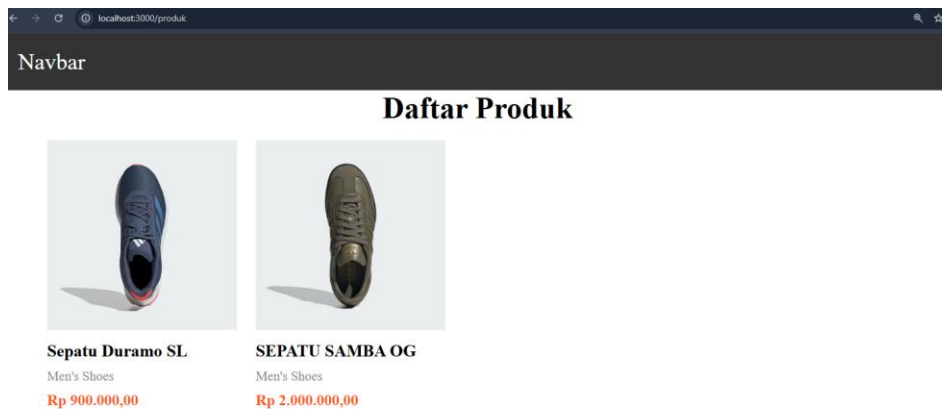
7. Modifikasi produk.module.scss

```
7.praktikum-Client rendering > my-app > src > pages > produk > product.module.scss > .produk > &__content > &__item
1  .produk {
2    width: 100%;
3    padding: 0 5%;
4    &__title {
5      text-align: center;
6      font-size: 32px;
7      font-weight: bold;
8      margin-bottom: 16px;
9    }
10   &__content {
11     display: flex;
12     flex-wrap: wrap;
13     gap: 20px;
14     &__item {
15       &__image {
16         width: 100%;
17         height: 200px;
18         object-fit: cover;
19         border-radius: 4px;
20         margin-bottom: 12px;
21       }
22       &__name {
23         font-size: 18px;
24         font-weight: 600;
25         margin-bottom: 8px;
26       }
27       &__price {
28         font-size: 16px;
29         color: #ff5722;
30         font-weight: bold;
31       }
32       &__category {
33         font-size: 14px;
34         color: #888;
35         margin-bottom: 8px;
36       }
37     }
38   }
39 }
40
```

8. Modifikasi Pada file index.tsx pada folder pages/views/product

```
7.praktikum-Client rendering > my-app > src > pages > views > product > index.tsx > ...
1  import styles from "../../produk/product.module.scss";
2
3
Windsurf: Refactor | Explain
4  type ProductType = {
5    id: string;
6    name: string;
7    price: number;
8    image: string;
9    category: string;
10 };
11
Windsurf: Refactor | Explain | Generate JSDoc | X
12 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
13   return (
14     <div className={styles.produk}>
15       <h1 className={styles.produk__title}>Daftar Produk</h1>
16       <div className={styles.produk__content}>
17         {products.map((products: ProductType) => (
18           <div key={products.id} className={styles.produk__content__item}>
19             <div className={styles.produk__content__item__image}>
20               <img src={products.image} alt={products.name} width={200} />
21             </div>
22             <h4 className={styles.produk__content__item__name}>
23               {products.name}
24             </h4>
25             <p className={styles.produk__content__item__category}>
26               {products.category}
27             </p>
28             <p className={styles.produk__content__item__price}>
29               Rp {products.price.toLocaleString()}
30             </p>
31           </div>
32         ))}
33       </div>
34     </div>
35   );
36 };
37
38 export default TampilanProduk;
--
```

9. Jalankan Browser



◆ Bagian 3 – Implementasi Skeleton Loading

- Modifikasi file index.tsx pada folder views/product/index.tsx

```
7.praktikum-Client_side rendering > my-app > src > pages > views > product > index.tsx > ...
1  import styles from "../../produk/product.module.scss";
   Windsurf: Refactor | Explain
2  type ProductType = {
3    id: string;
4    name: string;
5    price: number;
6    image: string;
7    category: string;
8  };
9
   Windsurf: Refactor | Explain | Generate JSDoc | X
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk__title}>Daftar Produk</h1>
14       <div className={styles.produk__content}>
15         { /* {products.map((products: ProductType) => (
16           <div key={products.id} className={styles.produk__content__item}>
17             <div className={styles.produk__content__item__image}>
18               <img src={products.image} alt={products.name} width={200} />
19             </div>
20             <h4 className={styles.produk__content__item__name}>
21               {products.name}
22             </h4>
23             <p className={styles.produk__content__item__category}>
24               {products.category}
25             </p>
26             <p className={styles.produk__content__item__price}>
27               Rp {products.price.toLocaleString()}
28             </p>
29           </div>
30         )} */ }
31         <div className={styles.produk__content__skeleton}>
32           <div className={styles.produk__content__skeleton__image}></div>
33           <div className={styles.produk__content__skeleton__name}></div>
34           <div className={styles.produk__content__skeleton__category}></div>
35           <div className={styles.produk__content__skeleton__price}></div>
36         </div>
37       </div>
38     </div>
39   );
40 };
41 export default TampilanProduk;
```

- Modifikasi file product.module.scss


```
1  .produk {
2    width: 100%;
3    padding: 0 5%;
4    &__title {
5      text-align: center;
6      font-size: 32px;
7      font-weight: bold;
8      margin-bottom: 16px;
9    }
10   &__content {
11     display: flex;
12     flex-wrap: wrap;
13     gap: 20px;
14     &__skeleton {
15       width: 200px;
16       padding: 16px;
17       border: 1px solid #eee;
18       border-radius: 8px;
19       box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
20       display: flex;
21       flex-direction: column;
22       align-items: center;
23       animation: identifier 1.5s infinite ease-in-out;
24       &__image {
25         width: 100%;
26         height: 200px;
27         background-color: #e0e0e0;
28         border-radius: 4px;
29         margin-bottom: 12px;
30       }
31       &__name {
32         width: 80%;
33         height: 20px;
34         background-color: #e0e0e0;
35         border-radius: 4px;
36         margin-bottom: 8px;
37       }
38       &__category {
39         width: 60%;
40         height: 16px;
41         background-color: #e0e0e0;
42         border-radius: 4px;
43         margin-bottom: 8px;
44       }
45       &__price {
46         width: 40%;
47         height: 18px;
48         background-color: #e0e0e0;
49         border-radius: 4px;
50       }
51     }
52   }
53   &__item {
54     &__image {
55       width: 100%;
56       height: 200px;
57       object-fit: cover;
58       border-radius: 4px;
59       margin-bottom: 12px;
60     }
61     &__name {
62       font-size: 18px;
63       font-weight: 600;
64       margin-bottom: 8px;
65     }
66     &__price {
67       font-size: 16px;
68       color: #ff5722;
69       font-weight: bold;
70     }
71     &__category {
72       font-size: 14px;
73       color: #888;
74       margin-bottom: 8px;
75     }
76   }
77 }
78 }
79
80 @keyframes identifier {
81   0% {
82     opacity: 1;
83   }
84   50% {
85     opacity: 0;
86   }
87   100% {
88     opacity: 1;
89   }
90 }
```

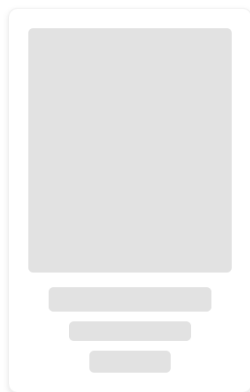
- Jalankan browser maka akan muncul skeleton yang terdapat animasi berkedip

Navbar

Daftar Produk

Navbar

Daftar Produk



- Modifikasi pada index.tsx pada folder views/product/index.tsx

```
7.praktikum-Client_side rendering > my-app > src > pages > views > product > index.tsx > ...
1  import styles from "../../produk/product.module.scss";
   Windsurf: Refactor | Explain
2  type ProductType = {
3    id: string;
4    name: string;
5    price: number;
6    image: string;
7    category: string;
8  };
9
   Windsurf: Refactor | Explain | Generate JSDoc | X
10 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
11   return (
12     <div className={styles.produk}>
13       <h1 className={styles.produk_title}>Daftar Produk</h1>
14       <div className={styles.produk_content}>
15         {products.length > 0 ? (
16           <>
17             {products.map((products: ProductType) => (
18               <div key={products.id} className={styles.produk_content_item}>
19                 <div className={styles.produk_content_item_image}>
20                   <img src={products.image} alt={products.name} width={200} />
21                 </div>
22                 <h4 className={styles.produk_content_item_name}>
23                   {products.name}
24                 </h4>
25                 <p className={styles.produk_content_item_category}>
26                   {products.category}
27                 </p>
28                 <p className={styles.produk_content_item_price}>
29                   Rp {products.price.toLocaleString()}
30                 </p>
31               </div>
32             ))}
33           </>
34         ) : (
35           <div className={styles.produk_content_skeleton}>
36             <div className={styles.produk_content_skeleton_image}></div>
37             <div className={styles.produk_content_skeleton_name}></div>
38             <div className={styles.produk_content_skeleton_category}></div>
39             <div className={styles.produk_content_skeleton_price}></div>
40           </div>
41         )}
42       </div>
43     </div>
44   );
45 };
46 export default TampilanProduk;
```

- Jalankan browser
 - Jika dijalankan akan muncul skeletonya terlebih dahulu setelah itu muncul gambar dan informasinya

◆ Bagian 5 – Implementasi SWR

<https://swr.vercel.app/>

1. Install SWR

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\7.praktikum-Client_side rendering\my-app> npm install swr

added 3 packages, and audited 429 packages in 2s

142 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

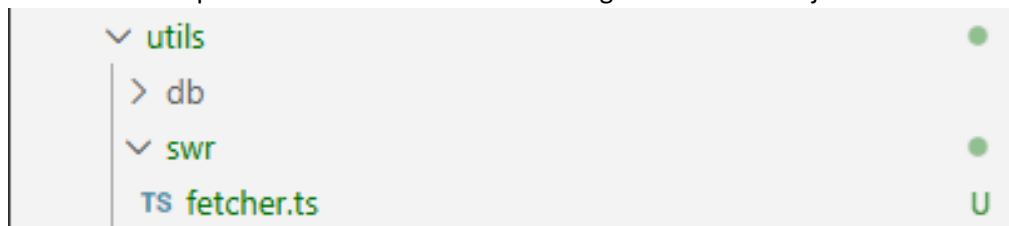
2. Buka dan modifikasi file index.tsx pada folder pages/product/

```
7.praktikum-Client_side rendering > my-app > src > pages > produk > index.tsx > kategori

1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3  import TampilanProduk from "../views/product";
4  import useSWR from "swr";
5
6  const fetcher = (url: string) => fetch(url).then((res) => res.json());
7  Windsurf: Refactor | Explain | Generate JSDoc | X
8  const kategori = () => {
9    // const [isLogin, setIsLogin] = useState(false); ...
10   const [products, setProducts] = useState([]);
11   // console.log("products:", products); ...
12
13   const { data, error, isLoading } = useSWR("/api/produk", fetcher);
14   //cek apakah data, error, dan isLoading sudah benar
15   // console.log("data:", data);
16   // console.log("error:", error);
17   // console.log("isLoading:", isLoading);
18
19   return (
20     <div>
21       <TampilanProduk products={isLoading ? [] : data.data} />
22     </div>
23   );
24 };
25
26 export default kategori;
```

3. Agar terlihat lebih rapi

- Buat folder swr pada utils dan tambahkan file dengan nama fetcher.js



- Modifikasi file fetcher.ts

7.praktikum-Client_side rendering > my-app > src > pages > utils > swr > TS fetcher.ts > [🔗] default

```
1
2  const fetcher = (url: string) => fetch(url).then((res) => res.json());
3
4  export default fetcher;
```

- Modifikasi file index.tsx pada folder pages/produk

7.praktikum-Client_side rendering > my-app > src > pages > produk > index.tsx > ...

```
1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3  import TampilanProduk from "../views/product";
4  import useSWR from "swr";
5  import fetcher from "../utils/swr/fetcher";
6
7
8
9  // const fetcher = (url: string) => fetch(url).then((res) => res.json());
10 const kategori = () => {
11   // const [isLogin, setIsLogin] = useState(false);
12   // const { push } = useRouter();
13   const [products, setProducts] = useState([]);
14   // console.log("products:", products); ...
15
16   const { data, error, isLoading } = useSWR("/api/produk", fetcher);
17   //cek apakah data, error, dan isLoading sudah benar ...
18
19
20   return (
21     <div>
22       <TampilanProduk products={isLoading ? [] : data.data} />
23     </div>
24   );
25 };
26
27 export default kategori;
```

Bandingkan:

- useEffect manual
- SWR

Note : untuk client side rendering dapat menggunakan 2 cara yaitu menggunakan manual (useEffect) atau menggunakan SWR

D. Tugas Praktikum

Tugas Individu

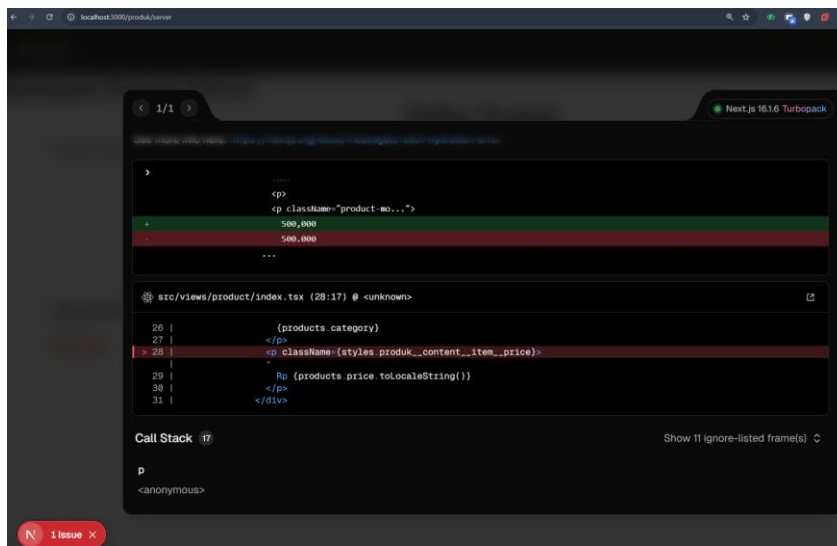
1. Jelaskan perbedaan:
 - Client Side Rendering
 - Server Side Rendering
 - Static Site Generation
2. Buat halaman produk dengan:

- Skeleton loading
 - Animasi
3. Refactor kode dari useEffect menjadi SWR.

Note :

Error

Jika saat menjalankan <http://localhost:3000/produk/server> muncul error sbb



Maka buka file index.tsx pada folder views/product/



Modifikasi pada code berikut

```
<p className={styles.produk_content_item_price}>
  Rp {products.price.toLocaleString()}
</p>
```

Menjadi :

```
<p className={styles.produk_content_item_price}>
  Rp {products.price.toLocaleString("id-ID")}
</p>
```

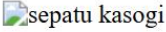
Jalankan <http://localhost:3000/produk/server>

← → ↻ ⓘ localhost:3000/produk/server

Navbar

Halaman Produk Server

Daftar Produk

sepatu kasogi



sepatu kasogi
sepatu lain
Rp 500.000

Sepatu Duramo SL
Men's Shoes



SEPATU SAMBA OG
Men's Shoes
Rp 2.000.000,00